Rapport de Projet : Optimisation pour l'Apprentissage Machine

Régression Logistique et Méthodes Proximales

Étudiant : [Namy AHmed C25394]

12 janvier 2026

Table des matières

1 Introduction **2**

2 Phase 1 : Fondements et Gradient Déterministe **2**
2.1 Analyse Mathématique 2
2.2 Calcul du Gradient et Constante de Lipschitz 2
2.3 Implémentation 3

3 Phase 2 : Passage à l'Échelle Stochastique **3**
3.1 SGD (Stochastic Gradient Descent) 3
3.2 Optimiseurs Modernes (RMSProp et Adam) 3
3.3 Analyse de l'impact du Momentum 3

4 Phase 3 : Non-Lissé, Parcimonie et Proximal **4**
4.1 Théorie : Opérateur Proximal 4
4.2 Algorithmes ISTA et FISTA 4
4.3 Interprétation 4

5 Conclusion **4**

1 Introduction

L'objectif de ce projet est d'explorer les fondements de l'optimisation numérique appliquée à un problème classique d'apprentissage supervisé : la classification binaire par régression logistique. Ce travail traverse trois paradigmes majeurs de l'optimisation moderne :

1. **L'optimisation convexe lisse (Phase 1) :** Utilisation de méthodes de premier ordre (Descente de Gradient) sur une fonction régulière.
2. **L'optimisation stochastique (Phase 2) :** Adaptation aux grands jeux de données (*Big Data*) où le calcul du gradient complet est coûteux.
3. **L'optimisation non-lisse et proximale (Phase 3) :** Introduction de la parcimonie via la régularisation L_1 (Lasso), nécessitant des méthodes proximales.

Nous travaillons avec un jeu de données $\{(x_i, y_i)\}_{i=1}^n$ où $x_i \in \mathbb{R}^d$ et $y_i \in \{-1, 1\}$.

2 Phase 1 : Fondements et Gradient Déterministe

Nous cherchons à minimiser la fonction de perte logistique avec régularisation Ridge (L_2) :

$$F(w) = \frac{1}{n}\sum_{i=1}^{n} \log(1 + e^{-y_i x_i^T w}) + \frac{\lambda}{2}\|w\|^2 \tag{1}$$

2.1 Analyse Mathématique

Justification de la classe C^2, convexité et forte convexité :

— **Classe C^2 :** La fonction $F(w)$ est une somme de fonctions composées. La fonction $u \mapsto \log(1+e^u)$ est infiniment dérivable sur $\mathbb{R}$. Le terme de régularisation $\|w\|^2$ est un polynôme quadratique, donc C^∞. Par somme et composition, F **est** C^2.

— **Convexité :** La fonction $f(z) = \log(1 + e^z)$ est convexe (sa dérivée seconde est la variance d'une sigmoïde, toujours positive). Comme $z = -y_i x_i^T w$ est une fonction linéaire de w, la composition reste convexe. La norme L_2 au carré est également convexe. La somme de fonctions convexes est convexe.

— **λ-forte convexité :** Le terme de régularisation $R(w) = \frac{\lambda}{2}\|w\|^2$ a pour Hessienne $\nabla^2 R(w) = \lambda I_d$. Les valeurs propres sont toutes égales à $\lambda > 0$. Par conséquent, $F(w)$ est au moins λ-fortement convexe.

2.2 Calcul du Gradient et Constante de Lipschitz

Calcul du gradient $\nabla F(w)$: En appliquant la règle de la chaîne, on obtient :

$$\nabla F(w) = -\frac{1}{n}\sum_{i=1}^{n} \frac{y_i x_i}{1 + e^{y_i x_i^T w}} + \lambda w \tag{2}$$

Ou, en utilisant la fonction sigmoïde $\sigma(z) = \frac{1}{1+e^{-z}}$:

$$\nabla F(w) = \frac{1}{n}\sum_{i=1}^{n} (\sigma(-y_i x_i^T w) - 1)(-y_i x_i) + \lambda w$$

[width=0.5]image.png

FIGURE 1 – Enter Caption

Constante de Lipschitz L : La Hessienne est majorée par $\frac{1}{4n}X^TX+\lambda I$. La constante de Lipschitz est donc :

$$L = \frac{1}{4n}\|X\|_{op}^2 + \lambda \tag{3}$$

Où $\|X\|_{op}$ est la plus grande valeur singulière de la matrice de données X.

2.3 Implémentation

- **Descente de Gradient (GD) :** On itère $w_{k+1} = w_k - \eta\nabla F(w_k)$ avec un pas fixe $\eta = 1/L$.
- **Gradient Conjugué :** Utilisé pour converger plus vite que la GD en utilisant des directions conjuguées pour éviter les oscillations .

3 Phase 2 : Passage à l'Échelle Stochastique

Lorsque n est très grand, le calcul du gradient complet est proscrit.

3.1 SGD (Stochastic Gradient Descent)

L'algorithme SGD estime le gradient sur un seul exemple (ou un petit batch) i_k choisi aléatoirement. **Mise à jour :**

$$w_{k+1} = w_k - \alpha_k \left(-\frac{y_{i_k} x_{i_k}}{1 + e^{y_{i_k} x_{i_k}^T w_k}} + \lambda w_k \right) \tag{4}$$

Règle de décroissance : Il faut un pas décroissant pour assurer la convergence, par exemple $\alpha_k = \frac{C}{\sqrt{k}}$.

3.2 Optimiseurs Modernes (RMSProp et Adam)

- **RMSProp :** Divise le gradient par une moyenne glissante de la magnitude des gradients récents.
- **Adam :** Combine l'idée de RMSProp avec le **Momentum**. C'est l'algorithme standard en Deep Learning.

3.3 Analyse de l'impact du Momentum

Le momentum ajoute une fraction de la mise à jour précédente :

$$v_{k+1} = \beta v_k - \alpha\nabla F(w_k) \quad \text{et} \quad w_{k+1} = w_k + v_{k+1}$$

Il permet de lisser les gradients stochastiques (réduction du bruit) et d'accélérer la convergence dans les directions de faible courbure.

4 Phase 3 : Non-Lissé, Parcimonie et Proximal

On remplace la régularisation L_2 par L_1 pour favoriser la sélection de variables (parcimonie) :

$$\min_w \Phi(w) = f(w) + \lambda \|w\|_1 \tag{5}$$

4.1 Théorie : Opérateur Proximal

La fonction $\|w\|_1$ n'étant pas dérivable en 0, on utilise l'opérateur proximal. La solution explicite est le **Seuil Doux (Soft Thresholding)** :

$$[S_\lambda(v)]_j = \text{sign}(v_j) \max(0, |v_j| - \lambda) \tag{6}$$

4.2 Algorithmes ISTA et FISTA

— **ISTA (Iterative Soft-Thresholding Algorithm) :** Descente de gradient sur f suivie d'une projection proximale.

$$w_{k+1} = S_{\lambda\eta}(w_k - \eta \nabla f(w_k))$$

— **FISTA (Fast ISTA) :** Version accélérée utilisant le momentum de Nesterov. Elle converge en $O(1/k^2)$ contre $O(1/k)$ pour ISTA.

4.3 Interprétation

Plus λ est grand, plus le seuil du Soft Thresholding est élevé, forçant un plus grand nombre de coefficients w_j à devenir exactement nuls. Cela effectue une sélection automatique des variables pertinentes.

5 Conclusion

Ce projet a permis de mettre en perspective les différentes approches d'optimisation :

1. Les méthodes de **Gradient Déterministe** sont précises mais coûteuses en calcul pour les grands n.
2. Les méthodes **Stochastiques (SGD, Adam)** permettent le passage à l'échelle au prix d'une convergence bruitée.
3. Les **Algorithmes Proximaux (ISTA/FISTA)** couplés à la norme L_1 permettent d'obtenir des modèles parcimonieux et interprétables.